Talking guide: disjointness & region analysis

What we added to `smash` and how to demo it

June 2026

What this is: notes for a conversation with collaborators — not a paper. Use it as a script: what we built, why it matters, what to try live, and what *not* to over-claim.

Contents

1 The one-sentence pitch

We taught `./smash` to read an ADL file and answer, for each pair of regions: *can the same event pass both?* or *are they forced apart?* — using heuristics plus optional Z3 proofs on the cuts we can extract.

2 Why we bothered

- Signal regions in one card often look related (SR1 vs SR6, etc.).
- Before this work, we only had flowcharts — no static check for overlap/disjointness.
- Useful for: combination studies, catching accidental double-counting, sanity-checking reinterpretation maps.

Scope we're honest about: single file only, and only the cuts the extractor understands. If half the cuts are weird BDTs, the tool will say so (coverage warnings).

3 What changed — quick timeline

Talk through this in order if people want the story:

1. **Make real ADL parse.** Example corpus went 49/68 → 68/68; fixed crashes on Delphes-style files.
2. **Object disjointness (`-r`).** "Are `bjets` and `electrons` the same kind of thing?"
3. **Region heuristics.** Interval clashes on `MET.pT`, `size(bjets)`, tags.
4. **Constraint IR + JSON.** Structured per-region facts for tooling.
5. **Z3 (SMT).** SAT = overlap proved (with witness); UNSAT = disjoint proved.
6. **Robustness pass.** Index-safe keys (`jets[0]` vs `jets[1]`), `size` as Int, `dPhi` in SMT.
7. **OR / ITE.** `A || B` and `size >= 3 ? dPhi(...) : ALL` like Delphes presel.

4 How to run it (demo script)

```
make
./smash -r myfile.adl                      # analysis + Z3 if installed
./smash -r --no-smt myfile.adl             # fast, heuristics only
./smash -r --json out.json myfile.adl      # machine-readable
make test-disjoint                         # small golden tests
```

Good demo files:

- `tests/golden/disjoint_pt.adl` — clear PROVEN DISJOINT
- `tests/golden/size_bjets.adl` — PROVEN OVERLAPPING (subset of multiplicity cuts)
- `examples/CMS/CMS-SUS-16-033_Delphes.adl` — real card, many SR pairs

5 What the output means

PROVEN DISJOINT

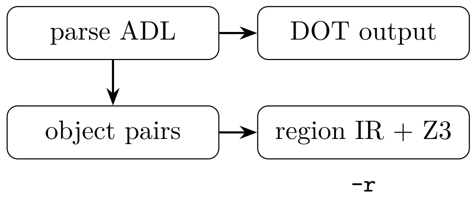
Intervals clash, or Z3 says UNSAT on $R_1 \wedge R_2$. Trust disjoint more than overlap — UNSAT is sound for what we encoded.

PROVEN OVERLAPPING
Z3 SAT + regions share a constraint dimension + witness printed. Means: *there exists* numbers satisfying both cut sets in our fragment. Not the same as "these SRs have yield in MC."

POSSIBLY OVERLAPPING
Heuristic only, or SAT on unrelated knobs (MET vs jet count).

UNKNOWN Extraction gap, timeout, or logic we don't encode yet.

6 Pipeline sketch (whiteboard version)



Main code: `semantic_checks.cpp` (extract cuts), `region_analysis.cpp` (pairs + Z3).

7 What we actually extract

Works well:

- Scalar cuts: `MET.pT > 200`, `pT(jets[0]) > 100`
- Multiplicity: `size(bjets) >= 2`
- Tags: `BTag == 1`
- Angular: `dPhi(jets[0], MHT) > 0.5`
- Inheritance: `select presel`
- `reject` (complemented)
- OR and Delphes-style ITE (v3)

Still weak / skipped:

- BDT scores, arbitrary functions
- Deeply nested OR/ITE
- Cross-file name alignment

Watch the line `N/M SMT-encodable` and coverage warnings — if it's low, don't treat Z3 verdicts as the full story.

8 Gotchas worth mentioning

- **Different jet indices are not disjoint.** `jets[0].pt > 300` and `jets[1].pt < 200` can both be true on one event. We fixed a bug where those were wrongly merged.
- **Subset ≠ disjoint.** `size >= 4` and `size >= 2` overlap heavily — Z3 correctly says PROVEN OVERLAPPING.
- **Z3 optional but recommended.** `apt install z3`; CI expects it.
- **Legacy verbose report** still exists: `-legacy-region-report` if someone wants the old wall of text.

9 Validation (if someone asks "is it tested?")

- 68/68 files under `examples/` parse and survive `-r`
- 7 golden ADL fixtures in `tests/golden/`
- GitHub Actions: build + golden + corpus + Delphes Z3 spike

10 What's next (only if the conversation goes there)

- Cross-file region comparison
- Richer SMT (nested logic, per-jet variables)
- Tie witnesses to actual CutLang execution / yields
- Clean up Bison grammar conflicts (separate chore)

11 Suggested 10-minute walkthrough

1. Show `./smash -r tests/golden/disjoint_pt.adl` — one PROVEN DISJOINT line.
2. Show `size_bjets.adl` — PROVEN OVERLAPPING + witness.
3. Open `CMS-SUS-16-033_Delphes.adl` — "this is a real card, not a toy."
4. Show JSON: `-json /tmp/x.json` and point at `fragment_coverage`.
5. Close with limits: fragment, not full simulation.

More detail in repo: `REGION_ANALYSIS.md`, `VERIFICATION_REPORT.md`, `docs/SMT_ROBUSTNESS_PLAN.md`.